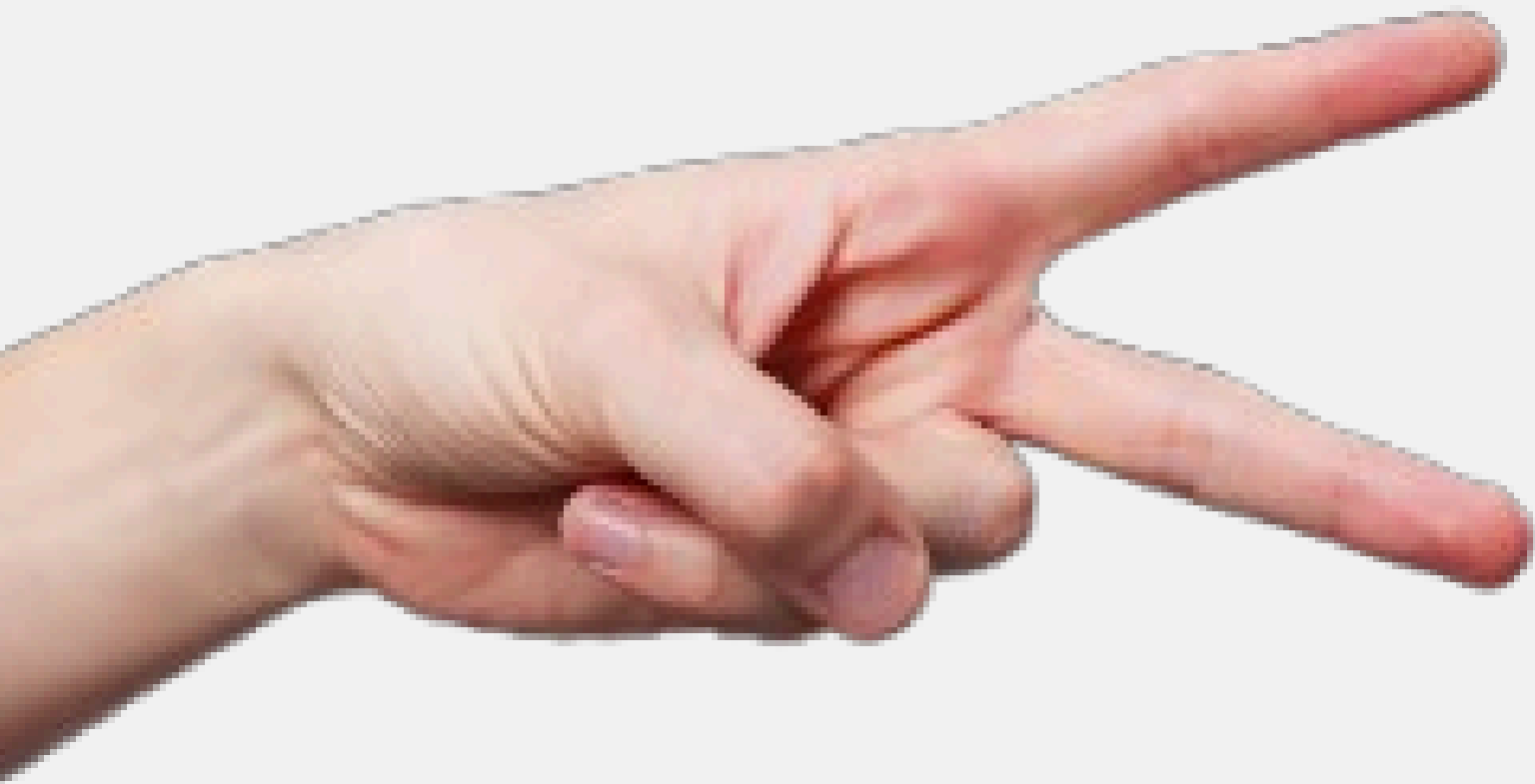


DOT the mind behind Tempo Comp
was created to help us.

Rock – Paper – Scissors



TEAM:

Antonio Brescia
Giuseppe D'Avanzo
Giorgio De Nigris



CHI SIAMO

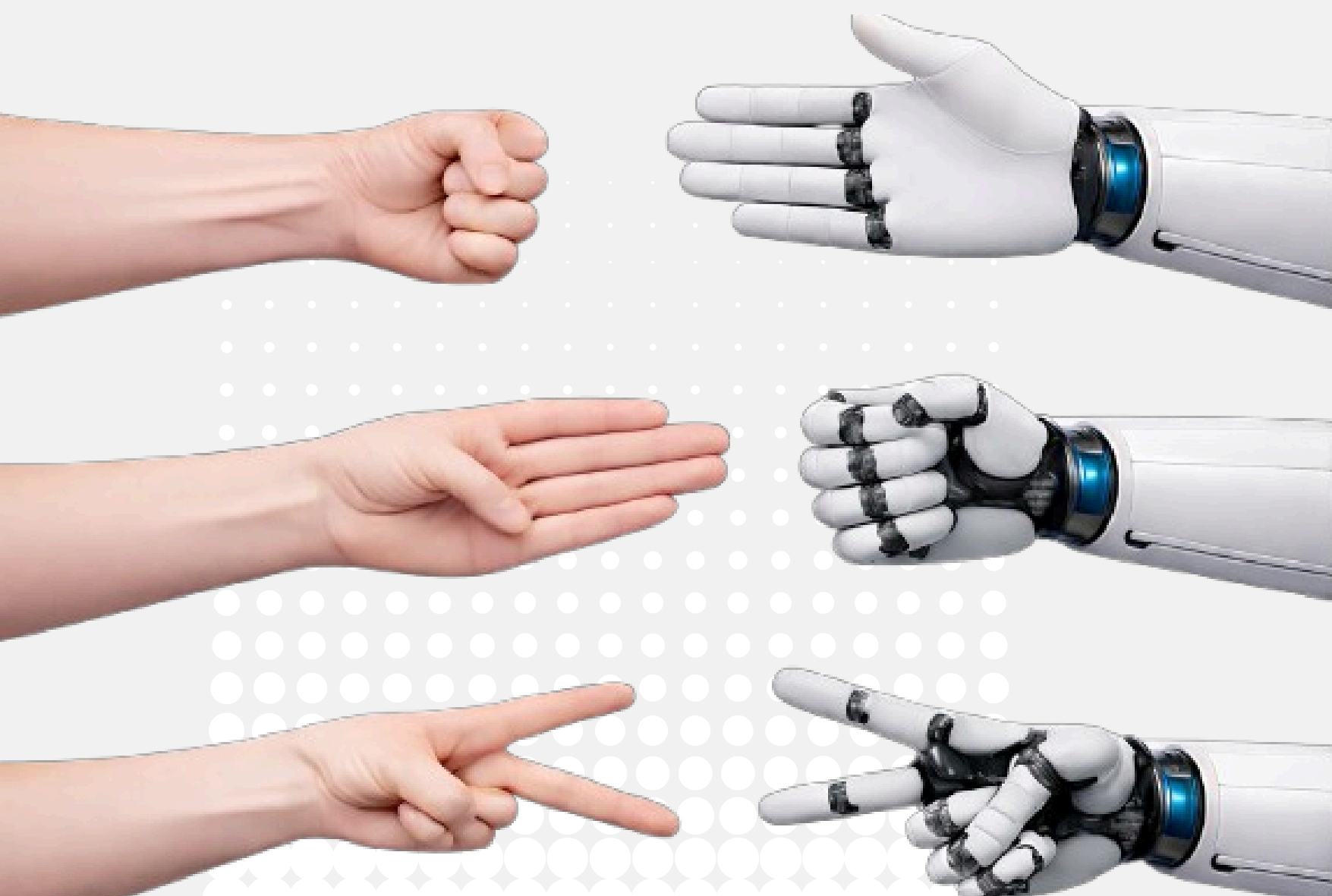


Giuseppe

Giorgio

Antonio

INTRODUZIONE



Descrizione del progetto

Realizzazione di un sistema in grado di riconoscere i gesti del gioco Rock-Paper-Scissors

Utilizzo della webcam per acquisire immagini

Classificazione in tempo reale tramite Deep Learning



Tecnologie utilizzate



Strumenti e librerie

Python

Utilizzato per lo sviluppo del modello e la gestione dei dati

Framework di Deep Learning

PyTorch

Usato per la creazione, l'addestramento e il testing della rete neurale

Modello

Convolutional Neural Network (CNN)

Per il riconoscimento di immagini e pattern visivi

Librerie principali

OpenCV

Acquisizione immagini da webcam e preprocessing

PyTorch

Gestione dataset e trasformazioni delle immagini

NumPy

Operazioni matematiche e gestione degli array



DATASET

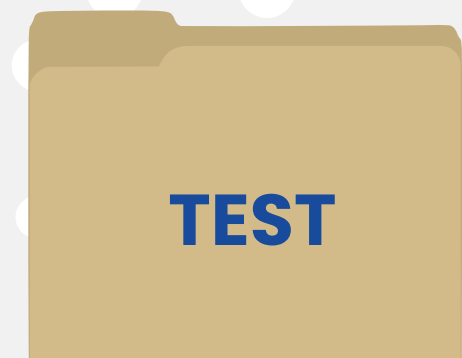
Composizione

3 classi:

Ogni immagine appartiene a una sola classe



Struttura



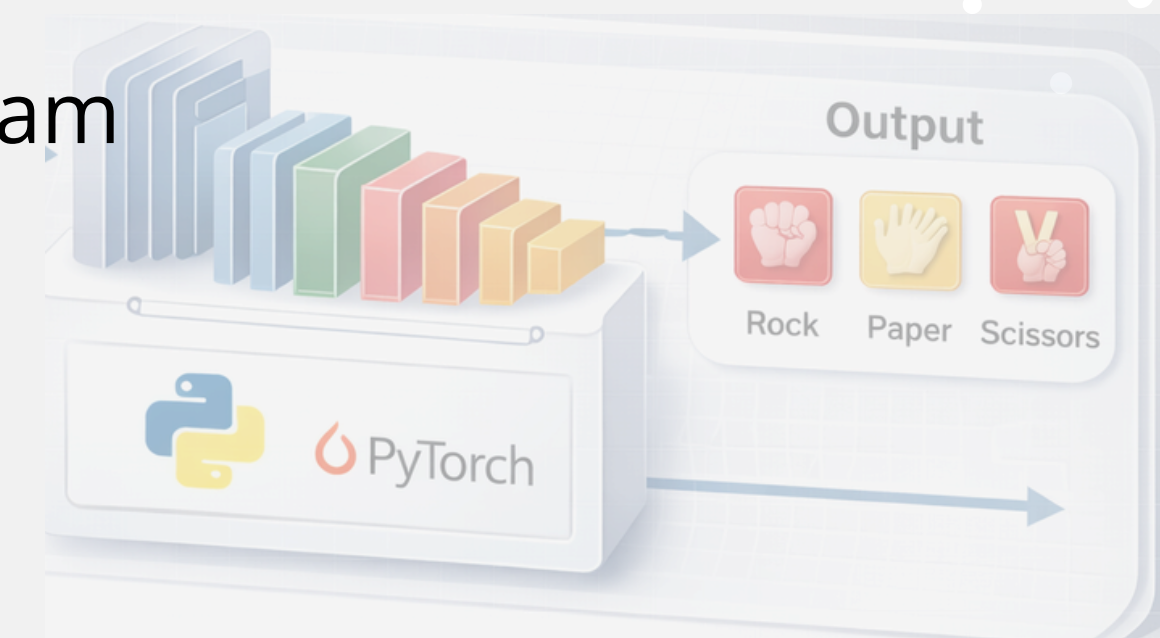
Modello di Deep Learning

RPSNet – CNN

Per il riconoscimento dei gesti Rock–Paper–Scissors è stato utilizzato un modello di Convolutional Neural Network (CNN) chiamato RPSNet

Utilizzo nel gioco

Modello addestrato su dataset di immagini
Caricato in modalità inferenza
Utilizzato in tempo reale tramite webcam



```
class RPSNet(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Conv2d(3, 32, 3),  
            nn.ReLU(),  
            nn.MaxPool2d(2),  
  
            nn.Conv2d(32, 64, 3),  
            nn.ReLU(),  
            nn.MaxPool2d(2),  
  
            nn.Flatten(),  
            nn.Linear(64 * 30 * 30, 128),  
            nn.ReLU(),  
            nn.Linear(128, 3)  
        )  
  
    def forward(self, x):  
        return self.net(x)
```

Addestramento e valutazione

Ottimizzazione dei pesi tramite

funzione di loss backpropagation

Training del modello

Addestramento supervisionato su immagini

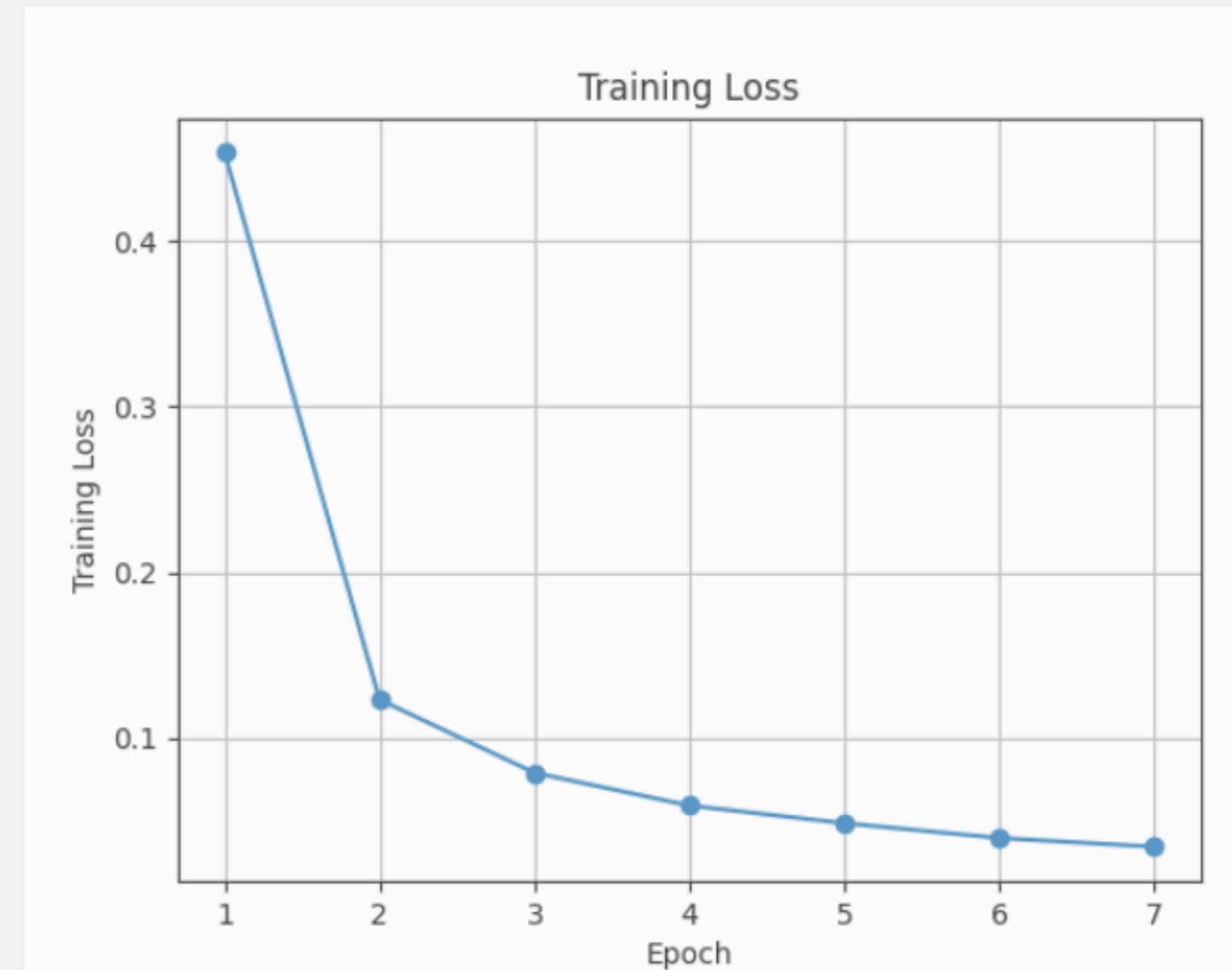
Valutazione Monitoraggio

Training Loss

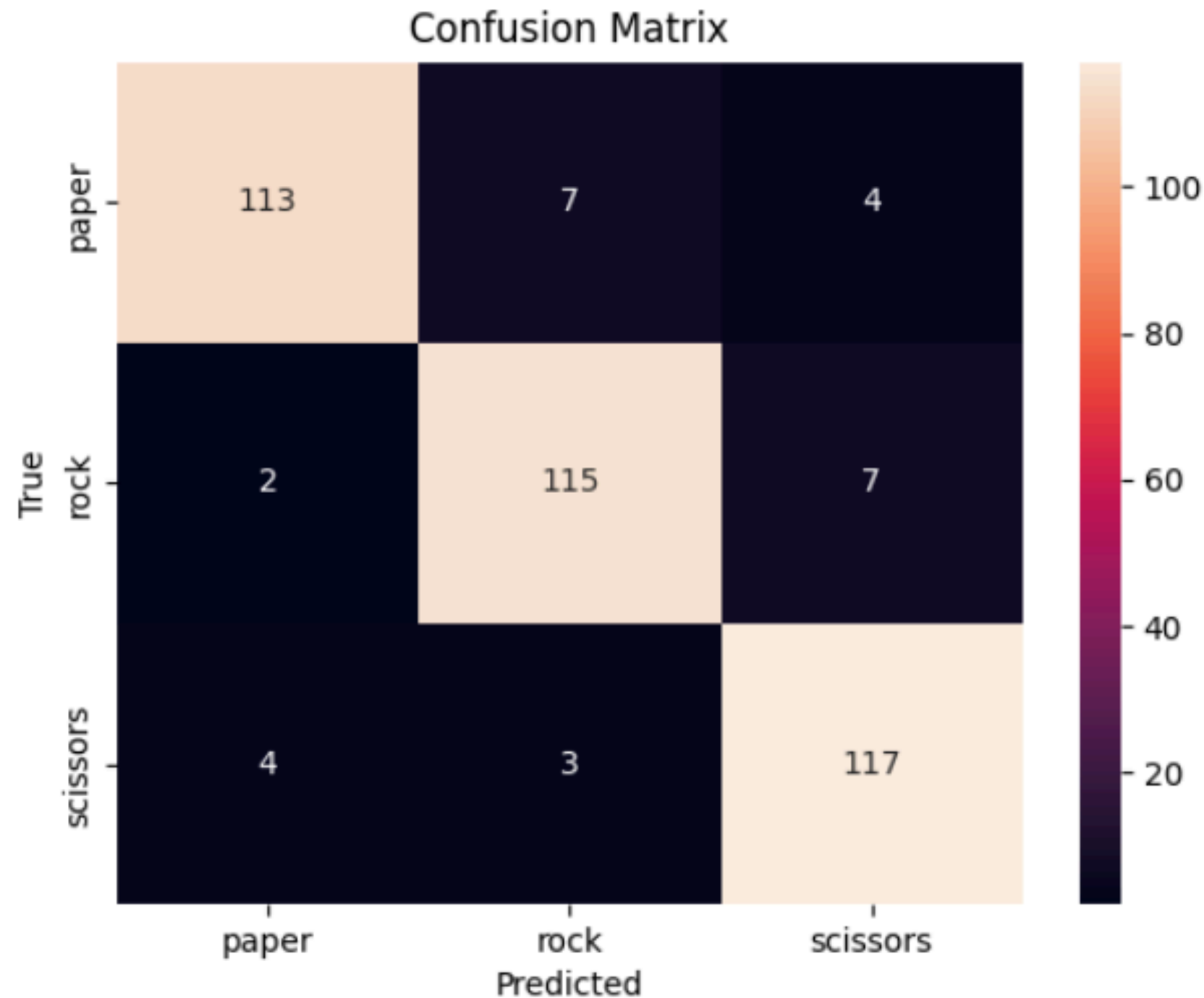
Validation Accuracy

Utilizzo di Early Stopping per evitare overfitting

Test finale su dati non visti



Valutazione del modello



CONSIDERAZIONI

La matrice di confusione mostra le prestazioni del modello sul test set

La maggior parte delle predizioni si trova sulla diagonale, indicando un'elevata accuratezza

Gli errori sono limitati e avvengono principalmente tra gesti visivamente simili

Nessuna classe risulta penalizzata in modo significativo

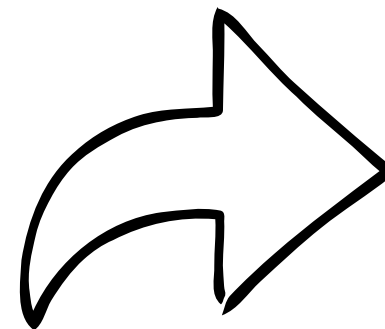
Problemi riscontrati

Gap tra accuracy di training e utilizzo reale
Sensibilità al background e all'illuminazione
Instabilità delle predizioni in tempo reale
Limitata varietà del dataset
Rischio di overfitting
Dipendenza dall'hardware disponibile



Soluzioni adottate / previste

Utilizzo di una Region of Interest (ROI)
→ riduzione del rumore di sfondo
Pre-processing con thresholding di Otsu
→ maggiore robustezza a variazioni di luce
Stabilizzazione tramite buffer temporale
→ voto di maggioranza sulle ultime predizioni
Unione di dataset sintetici e reali
→ aumento della varietà dei dati
Data augmentation
→ migliore generalizzazione del modello
Architettura CNN leggera
→ prestazioni real-time anche su hardware limitato



```
pygame.mixer.init(44100, -16, 2)

def gen_tone(freq, dur, vol=0.4):
    t = np.linspace(0, dur, int(44100*dur), False)
    wave = np.sin(freq * t * 2*np.pi)
    wave = (wave*(2**15-1)*vol).astype(np.int16)
    stereo = np.column_stack((wave, wave))
    return pygame.sndarray.make_sound(stereo)

snd_hit_ai      = gen_tone(220,0.12)
snd_hit_player  = gen_tone(90,0.15)
snd_win         = gen_tone(880,0.35)
snd_lose        = gen_tone(140,0.4)
snd_count       = gen_tone(600,0.08)
snd_click       = gen_tone(500,0.05)
```

Gestione del Sonoro Tramite
Procedural FX AUDIO

```
1 MAX_HP = 100
2 BASE_DMG = 25
3
4 def calc_damage(combo):
5     return BASE_DMG * 2 if combo >= 2 else BASE_DMG
6
```

Gestione HP e calcolo danni + Combo

```
CLASSES = ["rock", "paper", "scissors"]

if countdown == 0:
    ai_move = random.choice(CLASSES)
    win = decide(locked, ai_move)

    last_result = win
    result_time = time.time()

    if win == "PLAYER":
        p_combo += 1
        a_combo = 0
        ai_hp = max(0, ai_hp - calc_damage(p_combo))
        snd_hit_ai.play()
        shake_t = flash_t = time.time()
        flash_color = (0,255,0)

    elif win == "AI":
        a_combo += 1
        p_combo = 0
        player_hp = max(0, player_hp - calc_damage(a_combo))
        snd_hit_player.play()
        shake_t = flash_t = time.time()
        flash_color = (0,0,255)

    else:
        p_combo = a_combo = 0

    if ai_hp == 0:
        p_score += 1
        snd_win.play()
        ai_hp = player_hp = MAX_HP

    if player_hp == 0:
        a_score += 1
        snd_lose.play()
        ai_hp = player_hp = MAX_HP
```

Condizione di Win Lose
e
Combattimento dell'ai

Timer + Risultato Hit o Miss

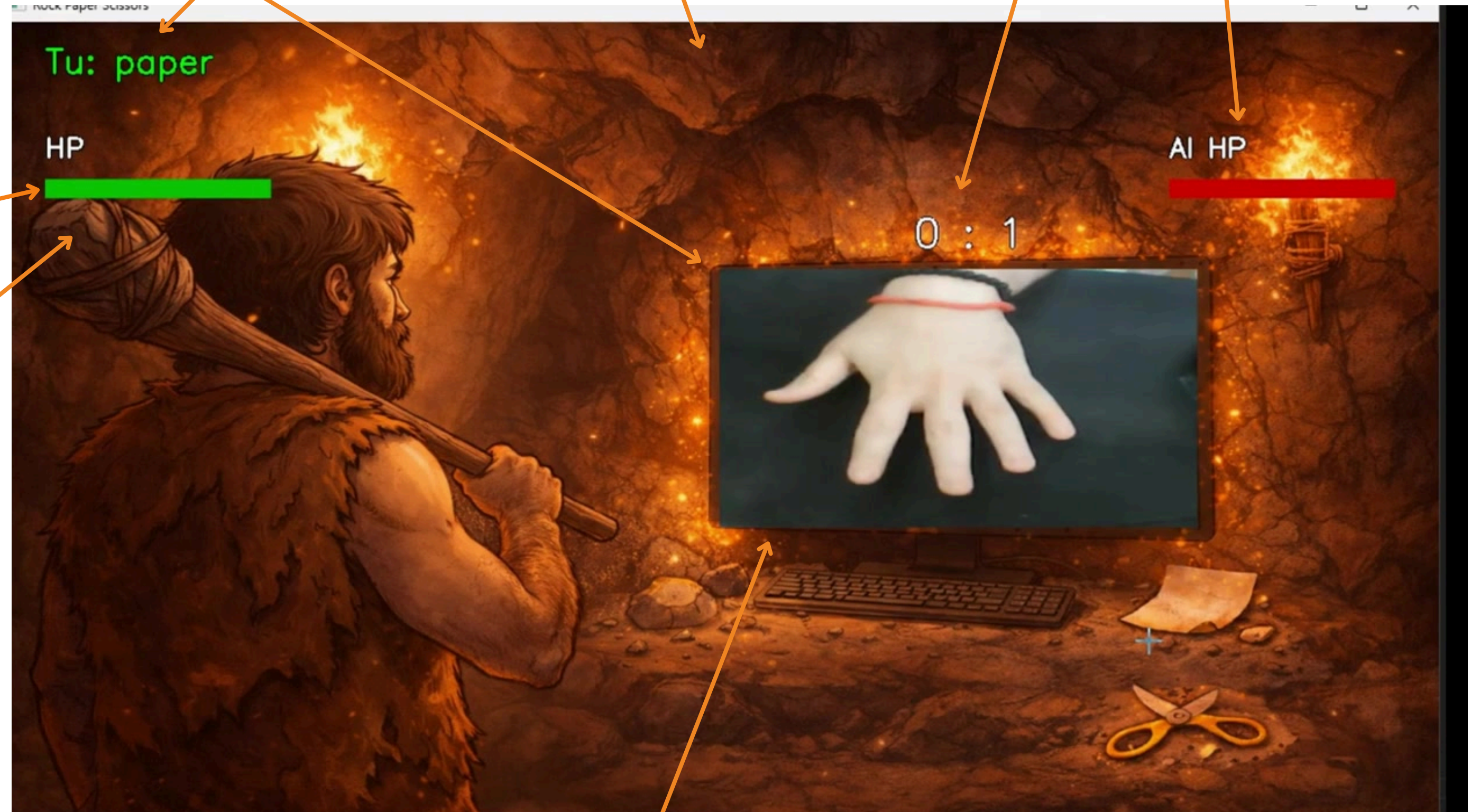
Riconoscimento della
mossa del nostro
giocatore

AI HP:
I punti vita dell'AI

Punteggi

HP:
I punti vita del nostro
Giocatore

Combo:
Danni inflitti x2



Cam del Giocatore

Grazie per aver giocato!

Game Design & Code

Giuseppe D'Avanzo

Giorgio De Nigris

AI

PyTorch CNN

Antonio Brescia

Ai Training

Giuseppe D'Avanzo

Giorgio De Nigris

Antonio Brescia



GAG – Rock Paper Scissors